Building data course — Week 4 — FDD at scale

sparql-playground / lessons

Generated for print and offline reading

Week 4 — FDD at scale

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. The scripts are in the repo next to each lesson. Keep notes under the ignored `student-work/` folder.

Contents

- Day 16 — One answer three ways: a Python loop, pandas and SQL

- Day 17 — Memory: lists, generators, DataFrames and chunks
- Day 18 — Gaps and coverage: sample-and-hold, three ways
- Day 19 — Joining weather: the as-of join
- Day 20 — Scaling showdown, and the Week 4 challenge

Day 16 — One answer three ways: a Python loop, pandas and SQL

Goal: compute the same fault count and run hours with a row-by-row Python loop, with vectorized pandas and with one SQL aggregate; see that they agree; then measure how each one grows with the data, and what moving the data costs.

Read

Week 4 asks one question over and over: *the answer is the same, so which tool should compute it?* You have written Open-FDD-style rules in SQL (Unit 1) and in pandas (Day 14). Plain Python is the third option, and the one a CS course starts with.

Today's job is deliberately small, so the three versions fit on a screen. Take the FC1 raw condition from Day 14 (**fan running and duct static more than 25 Pa below setpoint**) and compute two numbers:

- `fault_rows`: how many 5-minute rows meet the condition;
- `run_hours`: fan run hours, which is rows with `fan_status = 1` times 5 minutes.

The data is **synthetic** this week, so we can make it as big as we like. `uv run python lessons/course.py setup` wrote a million rows of two made-up AHUs (500,000 each, one row every 5 minutes, in SI units) to `var/synthetic_ahu.csv`, and the same rows to the `synthetic` table in `var/lessons.sqlite`. It has a daily fan schedule, duct static tracking its setpoint with noise, and a weak-fan window. Columns: `equipment_id`, `ts` (Unix seconds), `fan_status`, `static_pa`, `setpoint_pa`, `oat_c`. Both scripts take the **first n rows** (n = 10,000, 100,000 and 1,000,000), so they see identical data.

How each one does it:

	Plain Python (in `day16_pandas.py`)	**pandas (`day16_pandas.py`)**	**SQL (`day16_sql.py`)**
Where the rows live	Python lists pulled out of the columns	DataFrame columns (NumPy arrays)	table in the database
Who runs the loop	the interpreter, once per row	compiled C inside NumPy	compiled C inside SQLite
Condition	`if fan[i] == 1` and `if static[i] < setpoint[i] - TOLERANCE_PA`	`fan_on & too_low`	`fan_status = 1 AND static_pa < setpoint_pa - :tolerance_pa`
Add them up	`fault_rows = fault_rows + 1`	`.sum()`	`SUM(...)`

Big-O. All three are O(n): they touch every row once, so ten times the rows costs about ten times the time. Big-O hides the **constant factor**, and the constant is what you actually wait for. A Python loop pays interpreter overhead on every row: fetch a value,

compare, update a counter. NumPy and SQLite run the same logic in compiled loops, so their per-row cost is smaller by one to two orders of magnitude. Same curve, very different slope.

The hidden cost: moving data. The pandas and SQL numbers assume the data is already where that tool wants it: in a DataFrame, or in a table. If your data lives in SQLite and you want pandas, you first pay to copy it into a DataFrame (`read_sql_query`). If it lives in a CSV, you pay to parse it (`read_csv`). Those copies are O(n) too, and they are usually *bigger* than the computation they enable. Each script times the move it can see: the pandas script, reading the CSV; the SQL script, reading the table out into a DataFrame.

Run

```
uv run python lessons/course.py run 16
```

That runs `day16_pandas.py`, then `day16_sql.py` (a few seconds each). The pandas script has step 2, a plain Python loop, and step 3, the same count with pandas:

```
for i in range(n):
    if fan[i] == 1:
        run_rows = run_rows + 1
        if static[i] < setpoint[i] -
         TOLERANCE_PA:
            fault_rows = fault_rows + 1


fan_on = df["fan_status"] == 1
too_low = df["static_pa"] <
          df["setpoint_pa"] - TOLERANCE_PA
pandas_fault_rows = (fan_on &
          too_low).sum()
```

The SQL script moves the loop inside the database, and one row comes back:

```
SELECT SUM(fan_status = 1 AND static_pa <
          setpoint_pa - :tolerance_pa) AS
          fault_rows,
       SUM(fan_status = 1) *
          :hours_per_row AS run_hours
FROM synthetic
WHERE rowid <= :n
```

`SUM(fan_status = 1)` works because SQLite comparisons evaluate to 0 or 1. Both scripts print the same counts for every size (`fault_by_size` in `RESULTS`); the pandas script also checks the loop against pandas. The timings are **printed, never compared**: they differ on every machine and every run. On the machine this was written on:

```
rows: 10000
  fault rows: 1442
  run hours:  419.3
  loop gives the same answer: True
  seconds: read CSV 0.006 | loop 0.001 |
pandas 0.0004
rows: 100000
  fault rows: 14428
  run hours:  4164.0
  loop gives the same answer: True
  seconds: read CSV 0.032 | loop 0.006 |
pandas 0.0006
rows: 1000000
  fault rows: 133994
  run hours:  41664.0
  loop gives the same answer: True
  seconds: read CSV 0.291 | loop 0.067 |
pandas 0.0057
```

```
rows: 1000000
  fault rows: 133994
  run hours:  41664.0
  seconds: SQL aggregate 0.0946 | read
rows into pandas 1.506
```

Read the numbers in two directions. Down the sizes, the time grows about 10x per step once the data is big enough to matter: that's O(n). (The smallest size is dominated by fixed overhead: starting a query, allocating arrays.) Across one size, the tools differ by a constant factor. At a million rows the loop took about 0.07 s, the SQL aggregate about 0.09 s and pandas about 0.006 s, ten times faster than

either. Both pandas and SQLite are compiled loops, but pandas only has to compare and add whole arrays in memory, while SQLite evaluates the expression per row, decoding each row from its B-tree pages as it goes. (This loop is quick because it reads ready-made Python lists; Try it 1 shows what a loop over DataFrame rows costs.)

What did it cost to get the data there? Now the part that surprises people. Reading the CSV took 0.29 s, fifty times the pandas count. The SQL script's "read rows into pandas" is the move from a table into a DataFrame, with no computation at all: 1.5 s at a million rows, about 15 times the SQL aggregate that answers the question in place. Moving data costs one to three orders of magnitude more than the pandas arithmetic: reading a table out builds a Python tuple per row before converting to arrays, and parsing a CSV turns text back into numbers. If the data lives in SQLite and you need *one number*, "load into pandas, then compute" loses badly to the SQL aggregate, because the aggregate runs where the data is and only one row crosses the boundary. "pandas is faster than SQL" is only true *once the data is already a DataFrame*. If you will ask fifty questions of

the same data, pay the load once and use pandas. That trade-off is Day 20's main theme.

The worst version of all is the one that looks innocent: loop in Python, but over a list you built with `fetchall()`. You pay the transfer *and* the interpreter, and hold every row in memory as well. Day 17 measures that memory.

Try it

1. Change the loop to read DataFrame rows instead of lists: `for row in df.itertuples():` with `row.fan_status`, `row.static_pa`, `row.setpoint_pa`. Time it at 100,000 rows and say why it changed.
2. Add a third number to all three versions: the *fault hours*, `fault_rows * 5 / 60`. Which version needed the fewest new lines?
3. Time the opposite move: writing the DataFrame into a new SQLite table, `df.to_sql("copy", conn)` on an in-memory database (`sqlite3.connect(":memory:")`). How does it compare to the pandas count?

Answers

1. It gets about eight times slower (0.045 s against 0.006 s here). `itertuples` builds a tuple with *every* column of the row (and objects for the values you never use) before your `if` runs. Fetch only the columns you need. The same advice applies to `SELECT *` into a DataFrame, and it is why the course scripts use `usecols` when they read the CSVs.
2. SQL: one more expression, `SUM(fan_status = 1 AND static_pa < setpoint_pa - :tolerance_pa) * :hours_per_row AS fault_h`. pandas: one more line, `pandas_fault_rows * HOURS_PER_ROW`. The loop needs no new counter here (`fault_rows` already exists), but any number that isn't a function of the existing counters needs a new counter *and* a new `if`, and every extra number makes the loop longer.
3. About 1.5 s at a million rows: a few hundred times the pandas count. Inserting builds a B-tree page by page, and Python hands each row over one at a time.

Why it matters

Open-FDD ships every rule twice (pandas oracle, SQL pack) because they are better at different jobs, not because one is always right.

Today's tables are the first honest picture of that. Days 17–20 add the other axes: memory (Day 17), gaps in the data (Day 18), joining another time series (Day 19), and the full scaling showdown (Day 20).

Day 17 — Memory: lists, generators, DataFrames and chunks

Goal: measure how much memory the same rows cost as a list, as a stream and as a DataFrame; shrink the DataFrame with better dtypes; and aggregate a big table in memory proportional to a chunk, not to the table.

Read

Day 16 was about time. The other way a job fails is **memory**: the laptop swaps, or the container is killed. An FDD job over a year of 5-minute data on a hundred AHUs is tens of millions of rows. Ways to hold the rows:

Shape	What Python holds	Memory grows with
a `list` of tuples (`fetchall()`)	every row as a tuple of float objects	rows (and a lot of it per row)
a stream (`for row in cursor`, or `for row in df.itertuples()`)	one tuple at a time	nothing: O(1)

Shape	What Python holds	Memory grows with
a DataFrame (`read_csv`, `read_sql_query`)	NumPy arrays, one per column	rows x bytes per value
a chunked read (`chunksize=`)	one chunk of arrays at a time	the chunk size
a SQL aggregate	the answer (a couple of rows)	nothing: SQLite holds the data

A Python float is a 24-byte object, plus an 8-byte pointer in the tuple, plus the tuple's own header. A NumPy `float64` is 8 bytes, period. That's why DataFrames beat lists of tuples for numbers. But the DataFrame is still O(rows), and for text columns it's much worse: an `object` column stores a pointer per row to a separate Python string.

Two memory-saving moves, and one honest warning:

- **Right-size the dtypes.** A 0/1 status fits in `int8`. `float32` halves a column, but has only about 7 significant digits. That's fine for a pressure, risky for an epoch timestamp, and always something to *check* rather than assume. A text column with a few distinct values (`equipment_id`)

can be stored as `category`: small integer codes plus one lookup table.

- **Stream in chunks.** If the answer is an aggregate (counts, sums, a few rows per equipment), you never need the whole table in memory. Read a chunk, fold it into running totals, throw it away. Peak memory is O(chunk), and the result is the same as one big aggregate. Because sums and counts are *associative*, summing the chunk sums gives the total.
- **Warning:** the scripts measure memory with `tracemalloc` (start it, do the work, read the peak). It sees memory allocated by Python and by NumPy/pandas buffers. It does **not** see memory SQLite allocates inside its own C library (page cache, sort buffers). A SQL aggregate over 100,000 rows reports about zero because Python really did hold about nothing; SQLite's own footprint just isn't measured by it.

The task is the same as Day 16: the count of raw FC1 faults (fan running, static more than 25 Pa below setpoint), plus run hours, on the first 100,000 rows of the synthetic data (all AHU_1), identical in both scripts.

Run

```
uv run python lessons/course.py run 17
```

That runs `day17_pandas.py`, then `day17_sql.py`. The plain-Python ways of holding rows live in the pandas script; the SQL script is the aggregate, plus the cost of pulling the rows out of SQLite.

Step	`day17_pandas.py`	`day17_sql.py`
answer	3. `(fan_on & too_low).sum()`	1. `SUM(fan_status = 1 AND ...)`
rows as a list	1. `list(df.itertuples(index=False, name=None))`	2. `conn.execute(...).fetchall()` (transfer only)
rows as a stream	2. `for ... in df.itertuples(...)`	3. `for _ in conn.execute(...)` (transfer only)
whole DataFrame	3. `pd.read_csv(path, nrows=N, usecols=columns)`	4. `pd.read_sql_query(rows_sql, conn, ...)`
chunks	4. `pd.read_csv(..., chunksize=10_000)`, add up the pieces	5. `read_sql_query(..., chunksize=10_000)` (transfer only)

Step	`day17_pandas.py`	`day17_sql.py`
smaller dtypes	5. `astype({"fan_status": "int8", "static_pa": "float32", ...})`	(SQLite stores what it stores)

The pandas script prints (MiB of Python heap at the peak; the numbers vary a little by Python and pandas version):

```
AHU_1, first 100000 rows
  fault rows: 14428
  run hours:  4164.0
Peak Python memory (MiB)
  list of tuples: 11.45
  stream:         0.01
  whole DataFrame: 5.35
  chunked read:   1.5
DataFrame size (MiB): default dtypes 2.29
-> smaller dtypes 0.86
```

and the SQL script:

```
AHU_1, first 100000 rows
  fault rows: 14428
  run hours:  4164.0
Peak Python memory (MiB)
  SQL aggregate:  0.001
  fetchall list:  11.45
  cursor stream:  0.0
  whole DataFrame: 19.14
  chunked read:   2.55
```

Three things to see.

- **The stream is flat.** The loop holds one tuple, so its peak is about zero. Streaming trades speed for space (the interpreter loop is Day 16's slow one), but its memory cost is nothing.
- **A DataFrame's peak can be higher than its resting size.** `read_csv` parses straight into arrays (5.4 MiB). But `read_sql_query` has to fetch every row as Python tuples first and then convert them into column arrays, so for a moment both copies exist: 19 MiB, more than the 11 MiB list on its own. `chunksize` fixes that: about 2.5 MiB. Memory now scales with the **chunk**: double the table and the peak stays put (only the loop runs longer). The price is that this works for aggregates that can be folded (counts, sums, min/max). A rolling window or a streak that crosses a chunk boundary needs the chunks to

carry state across, which is what Day 18's loop does naturally.
- **The SQL aggregate is the cheapest in Python memory**, 0.001 MiB: Python holds a 1-row result while SQLite scans every row. But that number is incomplete: it's the Python side. SQLite's own memory (its page cache) is invisible to `tracemalloc`. To see it you would watch the process, for example with `psutil`'s resident set size.

Shrinking the DataFrame. Step 5 stores the flag as `int8` (1 byte instead of 8) and the two pressures as `float32` (4 instead of 8): 2.29 MiB becomes 0.86 MiB. Epoch timestamps would have to stay `int64` (they need 32+ bits, and `float32` would corrupt them). `float32` changes the *values* slightly (about 3e-5 at 350 Pa), so check that the fault decision survives (Try it 3). On data that sits *exactly on* a threshold, it could flip. Checking is how you find out, and why Open-FDD keeps one declared scale and dtype per column instead of guessing.

Try it

1. How big would the full DataFrame be for 100 AHUs x 1 year of 5-minute data (about 105,000 rows each) with five float64 columns plus an int64 `ts`? Work it

out from bytes per value, no new data needed.

2. The chunked read adds up `fault_rows` and `run_rows` per chunk. Why is *run hours* easy to fold across chunks, and what about a median duct static?
3. After step 5, count the fault rows again on `small` and compare with `fault_rows`. How many decisions flipped?

Answers

1. Bytes per row of the numeric columns, times rows:

```
numeric_bytes_per_row = 8 * 5 + 8  #
          five float64 + one int64
rows_total = 100 * 105_000
gib = rows_total *
          numeric_bytes_per_row / 2**30
          # about 0.47 GiB
```

   About 0.5 GiB for 10.5 million rows: fine on a laptop, but tens of GiB across a portfolio of buildings. That is where chunking, dtypes and “let the database aggregate” stop being optional.

2. Sums add: each chunk’s run rows plus the next chunk’s is the total, and hours = rows x 5 / 60 at the end. A *mean* you’d carry as (sum, count) and divide at the end; a median doesn’t fold this way at all.

3. None: `((small["fan_status"] == 1) & (small["static_pa"] < small["setpoint_pa"] - TOLERANCE_PA)).sum()` is also 14,428. The readings carry two decimals and the threshold is a round number, so float32's rounding never crosses it.

Why it matters

Memory decides where a rule can run. A rule that needs the whole table as a DataFrame fits a laptop but not an edge controller (Normal Framework on a Raspberry Pi 3 has under 1 GB for everything). Streaming and chunking are what let the same logic run in both places. Day 20 puts time and memory side by side.

Day 18 — Gaps and coverage: sample-and-hold, three ways

Goal: make a fault rule honest when the data has holes: hold each reading until the next one, treat a long gap as "unknown", confirm faults by elapsed time rather than row count, and report coverage next to fault hours. Implement it in SQL, pandas and plain Python, and compare all three.

Read

Day 14 counted rows and multiplied by 5 minutes. That works only if every 5-minute row exists. Real controllers go offline, networks drop, a trend log gets cleared. Open-FDD's rules (read `sql/fdd/postgres/rules/cmd1_fan_mismatch.sql`) handle this with **forward sample-and-hold**:

- A reading is assumed to hold until the *next reading arrives*. `held_seconds = next_ts - ts`. A missing row just means its predecessor holds for 10 minutes instead of 5, and the time is still counted.

- If the next reading is more than `MAX_GAP_HOURS` away (`MAX_GAP_SECONDS` in our scripts), we don't know what happened. `held_seconds = 0`: that stretch is **not fault time, not covered time**, and `gap_after = 1` marks the row so the streak **breaks** there. The last row of each equipment has no next reading, so `held_seconds = 0`.
- A fault is **confirmed** when its streak has lasted `CONFIRM_SECONDS` *of elapsed time*: `streak_elapsed = running sum of held_seconds within the streak`.
- Report `fault_hours = sum(held_seconds where confirmed) / 3600` and **coverage**: the seconds we could vouch for divided by the seconds in the window (first to last timestamp).

Why counting rows is wrong once there are gaps. Three things go wrong. (1) A dropped row silently removes 5 minutes of fault time. (2) Two fault rows on either side of a six-hour outage look like consecutive rows, so the streak "confirms" across time you never observed. (3) You cannot tell a healthy machine from one you couldn't see: `fault_hours = 0` with 40% coverage is not good news. Coverage keeps the answer honest.

The data is the synthetic AHUs from Day 16, cut to the first **4,000 rows** (about 14 days) of each AHU. Then the scripts punch holes in it, the way a flaky network does: every 37th row is dropped (a 10-minute gap), and on AHU_1 100 rows in a row are deleted (an outage of about 8 hours, while the fan is on). Both scripts build exactly the same table: 7,687 of 8,000 rows. The constants are at the top of each script: `MAX_GAP_SECONDS = 3600` (1 hour), `CONFIRM_SECONDS = 600` (the same 10 minutes as Day 14's two rows).

How each step maps across (compare Day 14's table):

Step	`day18_sql.py`	`day18_pandas.py`
4. next timestamp	`LEAD(ts) OVER (PARTITION BY equipment_id ORDER BY ts) - ts`	`df.groupby("equipment_id")["ts"].shift(-1)`
4. held / gap flag	`CASE WHEN seconds_to_next > :max_gap_seconds THEN 0 ...`	`seconds_to_next.where(~gap_after, 0)`

Step	day18_sql.py	day18_pandas.py
5. streak breaks	`LAG(raw_fault)` and `LAG(gap_after)`	`changed | after_gap | new_ahu`
6. elapsed in streak	`SUM(held_seconds) OVER (PARTITION BY equipment_id, streak_id ORDER BY ts)`	`groupby("streak_id")["held_seconds"].cumsum()`

The pandas script keeps both AHUs in one table, so a streak must also break at the first row of the next AHU (`new_ahu`); in SQL `PARTITION BY equipment_id` does that for free. SQLite has no `EXTRACT(EPOCH FROM ...)` like Postgres, but `ts` is already epoch seconds, so `next_ts - ts` is seconds directly. Units are SI: Pa, and seconds for time.

Run

```
uv run python lessons/course.py run 18
```

That runs `day18_pandas.py`, then `day18_sql.py`. `RESULTS` in both holds the per-AHU `summary` and the row-counting comparison (`naive_fault_hours`). Both print:

```
Rows left after the holes: 7687 of 8000

Gap-aware FC1 per AHU:
equipment_id  rows  long_gaps  fault_hours
covered_hours  window_hours  coverage
       AHU_1  3795          1       39.167
324.833        333.25     0.975
       AHU_2  3892          0       41.500
333.250        333.25     1.000

Row counting (Day 14) on the same rows:
equipment_id  naive_fault_hours
       AHU_1              38.33
       AHU_2              40.42
```

Look at what the gaps did. AHU_2 has only short drops, and sample-and-hold covers them: coverage 1.0. AHU_1's 8-hour outage is longer than the limit, so it is `long_gaps = 1` and coverage falls to 0.975 (covered time = window time minus the long gap: 333.25 − 8.42 = 324.83 h). The row before the outage has `held_seconds` 0 and the next row starts a new streak, so the fault episode that runs into the outage stops at the outage rather than stretching across it.

Now the wrong way. Step 8 runs Day 14's row-counting rule on the same gappy rows. It is low on both AHUs: 1.08 hours on AHU_2 and 0.83 on AHU_1. Every dropped row took 5 minutes of fault time with it, where sample-and-hold keeps them. On AHU_1 the row

count also has no notion of the outage: its streak runs straight across 8 unobserved hours. Only the gap-aware numbers come with a coverage figure that tells you how far to trust them.

A loop would be simple here too. One forward pass with four pieces of state (the previous raw flag, whether the previous row ended in a gap, the streak's elapsed time, the AHU) reads just like the description in "Read". SQL needs a window function per step. Day 20 writes that loop and times it.

Try it

1. Raise `MAX_GAP_SECONDS` to 12 hours (`12 * 3600`; `"max_gap_seconds"` in SQL). What happens to AHU_1's coverage, its `long_gaps` count and its fault hours?
2. Confirm faults over 1 hour (`CONFIRM_SECONDS = 3600`) instead of 10 minutes. Do fault hours rise or fall, and why?

Change the constant in both scripts and check that they still agree.

Answers

1. `long_gaps` 0 and coverage 1.0 for both AHUs, and AHU_1's fault hours jump

from 39.17 to 47.67: it now counts the 8 hours *as if the fault state at the start of the outage persisted through it*. That is the cost of a large gap limit: you assert knowledge you don't have. Pick the limit from how fast the equipment can change state, not from how much you'd like to cover.

2. Fault hours fall (AHU_1 39.17 to 35.58 h, AHU_2 41.50 to 38.17 h). Faults shorter than an hour never confirm, and in longer ones the first hour of each episode is not counted (a row only counts once its streak's elapsed time reaches the threshold). A longer confirmation time trades sensitivity for fewer false alarms.

Why it matters

Every real Open-FDD rule has this `elapsed_hold` block, and its parity tests check it. A rule without it will pass on clean test data and quietly over- or under-report on a building with a flaky gateway. Day 19 joins a second time series (weather) to these rows, and the same question (how old is too old?) returns.

Day 19 — Joining weather: the as-of join

Goal: attach the most recent weather reading to every 5-minute AHU row, ignoring readings older than two hours, in SQL (correlated subquery plus an index) and in pandas (`merge_asof`); see why the three ways to do it scale so differently; then write a sensor-versus-weather rule on top.

Read

Some Open-FDD rules need a number the building doesn't measure well: the real outdoor temperature. A sensor on an AHU's outside-air intake can sit in the sun, in a shaded louver, or be wired wrong. The `OAT-METEO` rule compares it with a weather feed (Open-FDD pulls hourly data from Open-Meteo) and flags a sensor that disagrees with the weather.

The join is awkward because the two series don't share timestamps. The AHU logs every 5 minutes; the weather arrives hourly and sometimes not at all. You can't `JOIN ... ON h.ts = w.ts`. You want, for each AHU row, **the latest weather row at or before that**

time, but only if it is recent enough (here: no more than 2 hours old; otherwise it is "stale", and the answer is `NULL`/`NaN`, meaning unknown, not "use whatever's old"). This is an **as-of join**.

The data. The AHU side is the synthetic AHU from Day 16: AHU_1's first 100,000 rows (about 347 days), with `oat_c`, its outdoor-air sensor in °C. **The weather is synthetic too**, in `var/synthetic_weather.csv` (and the `synthetic_weather` table): one row per hour, the same daily temperature wave as the AHU's sensor plus noise, with 12% of the hours deleted at random and six hours in a row missing near the start, the way a feed with failures looks. In production this table is the Open-Meteo feed. The AHU's sensor has a planted problem: for 100 rows in every week it reads **6 °C high**, like a sensor with the sun on it. The rule should notice.

How the join is done in each tool:

	SQL (`day19_sql.py`)	**pandas (`day19_pandas.py`)**	**plain Python (`day19_pandas.py`)**
Idea	per AHU row, look up the newest	merge two sorted tables in one pass	per AHU row, binary-search the

	SQL (day19_sql.py)	pandas (day19_pandas.py)	plain Python (day19_pandas.py)
	weather row <= ts		weather times
Code	(SELECT w.ts FROM synthetic_weather AS w WHERE w.ts <= h.ts AND w.ts >= h.ts - :max_age_seconds ORDER BY w.ts DESC LIMIT 1)	pd.merge_asof(ahu, weather, left_on="ts", right_on="weather_ts", direction="backward", tolerance=...)	bisect_right(weather_times, ts) - 1
Stale handling	AND w.ts >= h.ts - :max_age_seconds	tolerance= argument	if ts - weather_times[i] <= MAX_AGE_SECONDS

Complexity. Let N be AHU rows and M weather rows.

- A *nested loop* with no index: for each of N rows, scan all M weather rows. **O(N·M)**. SQLite does this when there is no index

on the weather `ts`; so does a naive Python double loop.

- *With an index* (SQLite) or a sorted list and binary search (Python's `bisect`): each lookup is O(log M), so **O(N log M)**.
- `merge_asof` on two *sorted* inputs walks both once: **O(N + M)**. The price: the inputs must be sorted by the key, which costs O(N log N) if they aren't.

Run

```
uv run python lessons/course.py run 19
```

That runs `day19_pandas.py`, then `day19_sql.py`. `ts` is epoch seconds in both tables. The pandas script prints:

```
AHU rows: 100000 | weather rows: 36586
Rows with no usable weather: 1372
Average weather temperature on the matched
rows (C): 9.967
bisect loop matched the same rows: True

Sensor vs weather rule (fan on, more than
3 C apart for 30 minutes):
  raw rows: 5089 | confirmed rows: 3892 |
hours: 324.33

Microseconds per AHU row (times vary by
machine):
  nested loop: 908.34
  bisect loop: 0.23
  merge_asof:  0.06
```

and the SQL script the same numbers, with its own timings and two query plans:

```
Microseconds per AHU row (times vary by
machine):
  no index:   900.47
  with index: 0.34
Plan on synthetic_weather: SEARCH
synthetic_weather USING COVERING INDEX
synthetic_weather_idx (ts>? AND ts<?)
Plan on weather_copy: SCAN weather_copy |
USE TEMP B-TREE FOR ORDER BY
```

`RESULTS` holds `rows_without_weather`, `average_weather_c` and the rule's counts; the unit test checks that both scripts agree.

The as-of join in SQL (steps 1-2). The inner query is run once *per AHU row*, which is what makes it "correlated". The weather value is then looked up by that timestamp with an ordinary join. In pandas, `merge_asof` needs both sides sorted by the key (they are). A detail worth knowing: both treat the 2-hour limit as inclusive, so a reading exactly 7,200 s old still counts. (The unit test would fail if they disagreed.)

How the database does it: `EXPLAIN QUERY PLAN` (Day 11). With the index, each probe is a B-tree range seek (`SEARCH ... USING COVERING INDEX`). On `weather_copy`, a copy with no index, it is a full scan of the weather table plus a temporary sort for the `ORDER BY`, for every AHU row. That is why the SQL script runs the no-index version on 200 rows only.

Plain Python, two ways (steps 3-4 of the pandas script). The nested loop is what SQLite does with no index. The `bisect` version is the same idea as the index: keep the weather timestamps sorted and binary-search them. The script prints `bisect loop matched the same rows: True`.

Timing. The timings are per AHU row, in microseconds, so different row counts compare fairly. The two O(N·M) methods take about 900 µs per row: at 100,000 rows that

would be 90 seconds, which is why they only get 200. With an index (or `bisect`, its Python twin) a lookup is a fraction of a microsecond: thousands of times faster, and the gap grows with M. `merge_asof` is the quickest because it does the whole job as one O(N + M) merge in compiled code, with almost no per-row overhead. SQLite's unindexed subquery is a compiled nested loop, but it is no faster than the Python one: the curve, not the language, decides.

The lesson that carries over to all of Week 4: an index (or a sort) changes the *shape* of the curve. Vectorizing only changes the slope.

A sensor-versus-weather rule (step 5). In the spirit of `OAT-METEO`: while the fan runs, flag the AHU when its outdoor sensor differs from the weather by more than 3 °C, confirmed for 6 rows (30 minutes). No weather (stale or missing) means *unknown*, so no fault. In SQL, the join result is a temp table under Day 14's streak steps; in pandas, the same `shift` / `cumsum` / `cumcount` trick. It finds 3,892 confirmed rows, 324 hours: the "sun on the sensor" stretches. In real data there would be more false alarms, which is why the rule has a 3 °C tolerance, a confirmation time and a stale-weather guard. Note what the 2-hour

limit bought: when the weather feed is down, the rule says nothing, instead of comparing a live sensor to yesterday's weather.

Try it

1. Tighten the limit to 1 hour (`MAX_AGE_SECONDS = 3600`, `"max_age_seconds"` in SQL). How many AHU rows lose their weather?
2. Replace "latest at or before" with "nearest *after*": SQL `w.ts >= h.ts AND w.ts <= h.ts + :max_age_seconds ... ORDER BY w.ts ASC`, pandas `direction="forward"`. When would you ever want that, and when is it a bug?

Answers

1. 11,530 rows have no usable weather, against 1,372 with 2 hours. With a 1-hour limit, every AHU row in the hour after a missing weather hour becomes unknown (12% of hours are missing, so about 12% of rows). The rule's confirmed hours drop to 259.5. The two implementations still agree.

2. In SQL, swap the condition and the sort in the subquery:

```
WHERE w.ts >= h.ts AND w.ts <= h.ts +
        :max_age_seconds
ORDER BY w.ts ASC LIMIT 1
```

Every matched weather timestamp is then at or after the row's (1,372 rows again have no match, the ones before a gap this time). The forward join pairs each row with a *future* weather reading. That is legitimate for offline analysis when a value is stamped at the *end* of its interval (a mean over the previous hour). It is a **look-ahead bug** in anything that claims to run live, or in a backtest of a live rule: the rule would "know" something it couldn't have known yet. Always ask which direction is honest for the data you're joining.

Why it matters

"Latest value at or before" is the most common join in building data: setpoints that change a few times a day against readings every minute, schedules against measurements, weather against everything. Remember the three shapes: a nested loop (O(N·M)), an index lookup (O(N log M)), a sorted merge (O(N + M)). Day 20 pits the three tools against each other on the FC1 rule at scale.

Day 20 — Scaling showdown, and the Week 4 challenge

Goal: run Day 14's FC1 confirmation-streak rule as a plain Python state machine, as vectorized pandas and as SQLite window SQL on data that grows 10x per step; compare the times and how they grow; and decide, with numbers, when each one is the right tool.

Read

This is the same rule as Day 14: raw fault = fan running and duct static more than 25 Pa below setpoint; a fault is **confirmed** once the raw fault has held for 2 consecutive rows. All the rule's cleverness is in "consecutive": each row's answer depends on the rows before it. That's the part that makes the tools differ.

	Plain Python (`day20_pandas.py`)	**pandas (`day20_pandas.py`)**	**SQLite (`day20_sql.py`)**
Idea	one pass, a tiny state machine:	column ops: `shift`, `cumsum`, `cumcount`	window functions: `LAG`, `SUM OVER`,

	Plain Python (`day20_pandas.py`)	**pandas (`day20_pandas.py`)**	**SQLite (`day20_sql.py`)**
	`streak_length`		`ROW_NUMBER`
State	one integer, carried along	none: streak ids are *recomputed* with `cumsum`	none: window frames recompute it
Work	1 pass, interpreter speed	~5 passes over whole arrays, C speed	sort/partition, several window passes
Memory	O(1) (streams the rows)	O(n) (needs all rows in arrays)	the database engine's (not in Python)
Data starts	in Python objects	has to be loaded first	where it already is

A state machine is the natural way to say "count consecutive rows" in a loop, and the awkward way to say it in column operations (Day 13 needed a trick: compare with the previous row, running-sum the changes). SQL and pandas are both doing that trick. Whether the trick costs more than the interpreter overhead is what the tables below show.

The data is Day 16's synthetic AHUs, the same rows in both scripts. The showdown uses AHU_1 only, so the loop never has to notice a change of AHU; the challenge at the end takes both AHUs, one at a time in pandas and with `PARTITION BY equipment_id` in SQL.

Run

```
uv run python lessons/course.py run 20
```

That runs `day20_pandas.py` (the loop and the pandas rule), then `day20_sql.py` (the window SQL). It is the longest pair of the week (several seconds each). `RESULTS` in both holds `totals_by_size` (confirmed rows and episodes at every size), `first_fault_ts` (the first confirmed fault) and `longest_episode` (the challenge below). The unit test compares them; you compare the timings. The pandas script prints:

```
rows: 1000 | fault rows: 286 | episodes: 2
  loop agrees: True
  seconds: loop 0.0001 | pandas 0.0014
rows: 10000 | fault rows: 1430 | episodes:
10
  loop agrees: True
  seconds: loop 0.0013 | pandas 0.0013
rows: 100000 | fault rows: 14300 |
episodes: 100
  loop agrees: True
  seconds: loop 0.012 | pandas 0.0059
rows: 500000 | fault rows: 70928 |
episodes: 496
  loop agrees: True
  seconds: loop 0.0588 | pandas 0.0343
```

and the SQL script the same counts with its own timing (on the machine this was written on):

```
rows: 100000 | fault rows: 14300 |
episodes: 100
  seconds: window SQL 0.2479
rows: 500000 | fault rows: 70928 |
episodes: 496
  seconds: window SQL 1.2679
```

Reading the timings. Times vary by machine and run, so they are printed, never compared. Down the sizes, about 10x per 10x of rows means O(n). Less than that at the small sizes means fixed overhead (starting a query, allocating arrays) is still a big part of the time; more than that would mean worse

than linear (sorting adds a log factor). The loop grows about 10x per step from the start; pandas barely grows at first (overhead), then about 5x per 5x; the window SQL about 10x per step.

At 500,000 rows: pandas on data already in memory 0.034 s, the plain loop 0.059 s, window SQL 1.27 s. Three readings of the same numbers:

- *If the data is already a DataFrame*, pandas wins, but only by about 2x over the loop. The loop's penalty is smaller than on Day 16's plain counting would suggest, because a state machine is *one* pass, while the pandas version needs several full-array passes (`shift`, `!=`, `cumsum`, `groupby(...).cumcount()`).
- *If the data lives in the database*, add Day 16's cost of moving it out (about 1.5 s for a million rows) to the pandas number before comparing; the loop pays a similar price to pull rows out of a cursor.
- *Window SQL is the slowest here*, about 35x pandas and 20x the loop: SQLite sorts the rows, then makes several window passes, each of which materializes intermediate rows (the script also copies the rows into temp tables `h` and `fc1` for each size). SQL's strength is not this kind of row-to-row logic. It is the *selective*

query that an index answers without reading most of the table (Day 11), and the aggregate that runs where the data is (Day 16).

So when does each one win?

- **SQL** when the data already lives in the database and you want a small answer: a total, a top-10, one AHU's last week. The index finds the rows without reading the rest (Day 11), nothing is copied into Python, and the result is one small table. Also when many people or programs need the same logic: the query ships with the data. It loses on cost-per-row for complicated row-to-row logic, and it is awkward to step through.
- **pandas** for in-memory column math and exploratory work: it is the fastest once the data is in arrays, and the interactive loop (change a line, look at a plot) is why it's the oracle in Open-FDD's parity tests. It loses when the data doesn't fit in memory (Day 17), or when moving the data in costs more than the calculation (Day 16).
- **A plain loop** when the data streams past once and the state is complicated or the data is small: it needs O(1) memory (Day 17), it can **stop early** (next), and a state machine with ten fields is easier to read than ten window functions. Its per-row

cost is high, but it is flat. If a loop is the right *shape* and merely slow, tools like numba or Cython compile exactly this kind of loop to machine code; that's a story for another course.

Open-FDD ships the pandas version as the reference and the SQL version where the data lives. Both are correct, and the parity test is what lets you pick per situation.

Try it

Stop early. Step 3 finds the *first* confirmed fault in AHU_1's first 200,000 rows: the loop stops (`break`) at the first hit, pandas and the window SQL compute every row first. All three find the same timestamp. Compare their times, then delete the `break` (so the loop reads the whole table) and compare again.

Answer

The scripts print:

```
First confirmed fault in 200,000 rows: ts
1773727500
  pandas finds the same one: True
  seconds: loop with early exit 0.0001
  seconds: pandas (computes every row
first) 0.0125
```

and `seconds: window SQL 0.4939`. The first confirmed fault is row 361 (ts 1773727500 is 108,300 s, about 30 hours, after the start), so the loop reads 361 rows of 200,000 and stops: about 100x faster than pandas, and thousands of times faster than SQL. pandas and the window SQL have no equivalent: they compute every row's state before you can ask for the first. `SELECT MIN(ts)` takes as long as the whole query (and a `LIMIT 1` would not help for the same reason). Without the `break` the loop reads all 200,000 rows, about twice the 100,000-row time above, and it is slower than pandas again. The loop's win is in *not needing the rest of the data at all*, which matters most when the answer is near the start or each row costs a network call.

Week 4 challenge

Find the **longest confirmed fault episode** for each AHU: its length in rows and in minutes, in the first 20,000 rows of each AHU. An episode is a maximal run of consecutive raw-fault rows that lasts at least `CONFIRM_ROWS` rows. Implement it three ways, and say which one you would ship if the data were (a) one building's database, (b) a notebook of a month of data, (c) a stream on a controller.

- **pandas** (step 4 in `day20_pandas.py`): number the streaks, get the size of each with `groupby("streak_id")`, keep the raw-fault ones that last long enough, take the max, for each AHU in turn.
- **SQL** (step 4 in `day20_sql.py`): `GROUP BY equipment_id, streak_id HAVING COUNT(*) >= :confirm_rows`, then `MAX`.
- **Loop** (your turn): a streaming loop that tracks the current and the best run.

Both scripts print the same table (`longest_episode` in `RESULTS`):

```
equipment_id  longest_rows  longest_min
       AHU_1           144          720
       AHU_2           144          720
```

Answer

Both AHUs have a longest episode of 144 rows = 12 hours: the 12-hour fan schedule (06:00-18:00) lies entirely inside the weak-fan window, so the whole run is one episode (synthetic data: a real building would be messier).

The key step in all three is the same trick as Day 14: a new streak starts where the raw flag changes, the running sum of starts numbers the streaks, and the size of a streak is its length. In pandas that is `(raw !=`

`raw.shift()).cumsum()`; in SQL `LAG` then `SUM() OVER`; in the loop, step 2a's `streak_length`, plus one more variable:

```
longest = 0
for i in range(n):
    # ... step 2a's lines that update raw
         and streak_length ...
    if raw and streak_length >=
         CONFIRM_ROWS and streak_length >
         longest:
        longest = streak_length
```

Which to ship: (a) one building's database: **SQL**, next to the data, with an index, returning two numbers. (b) a notebook with a month of data: **pandas**, because you'll want to plot the episodes next. (c) a stream on a controller: the **loop**, with O(1) memory and early updates the moment each episode ends. All three give the same answer, which is the point of having two scripts to compare.

Week 4 in one sentence: the answer is the same everywhere, and *where the data is, how much of it you hold in memory, and what shape the logic has* decide which tool gets there cheapest.

Reading — Week 4

The lessons first, then these for exact behavior.

- Python timeit
- Python tracemalloc
- pandas: scaling to large datasets
- pandas.merge_asof
- SQLite query optimizer overview
- Open-FDD SQL cookbook